

Test Report — Buzz Buzz & Populace

A record of what was verified on 12 August 2026, what could not be, and one defect found in our own tooling during the attempt. Written to be checkable: every figure below came from a command whose output is reproducible.

12 August 2026	Populace 0.1.0	jqepegeifmnofefeybrz	Partial
Date of testing	Test platform	Isolated test project	Overall outcome

1 • Summary

Seven of eight planned checks passed. The eighth — a live multi-user simulation against Buzz Buzz — **could not be completed**, because the test machine's network dropped roughly half of all outbound TLS connections for the duration. No result is reported for it, and no figure from the two aborted attempts has been published anywhere.

The attempt was not wasted. It exposed a genuine defect in Populace's `clean` command that had been present since the command was written, and which would have affected any customer on an unreliable connection. That defect is fixed, tested and released.

CHECK	RESULT	EVIDENCE
Populace self-test suite	PASS	24 of 24 checks
End-to-end demo run	PASS	Planted defect found; exit 1
Production safety guard	PASS	Refused live host; exit 1
Adapter contract coverage	PASS	13 of 13 methods
Test-project schema	PASS	16 of 16 tables created
Backend access control	PASS	0 rows to anonymous callers
Buzz Buzz build integrity	PASS	Type check clean; build 6.06 s
Live multi-user simulation	NOT RUN	Network unfit; see §4

2 • Product one — Buzz Buzz

2.1 Build integrity

TypeScript compilation completed with no errors. The production bundle built in 6.06 seconds. One advisory remains: a chunk exceeds 500 kB after minification. It is a warning, not a failure, and is recorded here rather than omitted.

2.2 Database schema

The consolidated schema (`00_complete_backend.sql` , 1,581 lines, 17 parts) was applied to a newly created, empty Supabase project. All **16 public tables** were created: `profiles`, `driver_settings`, `worker_locations`, `route_points`, `jobs`, `feed_posts`, `chat_threads`, `chat_thread_members`, `chat_messages`, `notifications`, `device_tokens`, `post_likes`, `post_comments`, `connections`, `groups`, `group_members`.

2.3 Access control

Probed as an unauthenticated caller holding only the publishable key — the same key that ships inside the Android package, and therefore the key an attacker is assumed to hold.

TABLE	ROWS RETURNED	INTERPRETATION
profiles	Access denied (42501)	Column-level grant holds; phone numbers unreadable
feed_posts	0	Row-level security filtering correctly
worker_locations	0	No GPS history exposed
chat_messages	0	No private messages exposed
route_points	0	No route history exposed

A status of 200 with zero rows is the correct outcome: the request succeeds and row-level security returns nothing. It is materially different from an error, and both were checked rather than assumed.

3 • Product two — Populace

3.1 Self-tests and end-to-end demo

The suite runs the entire product against an in-memory adapter, so a pass demonstrates that the engine carries no coupling to any real application. **24 of 24 checks passed.** The bundled demo, which runs against a fake app containing a deliberately planted row-level-security defect, found that defect and exited non-zero as designed.

```
✗like           41    10    43ms    60ms
  |, 10x new row violates row-level security policy "post_likes_insert"
✓ Cleanup complete — 6 accounts removed.      exit 1
```

3.2 Production safety guard

Populace was pointed deliberately at Buzz Buzz's **live production backend** to confirm it refuses. It stopped before loading the adapter:

```
✗ REFUSING TO RUN
The target matches a host listed in neverRunAgainst:
  https://ypdaetbeexyepswhybui.supabase.co
Simulated people must never be visible to real users.  exit 1
```

This is the safety property the product is sold on, demonstrated against the real host rather than asserted.

4 • The check that did not complete

A live simulation of six drivers across Manila and Mumbai was attempted twice and abandoned both times.

Cause. Measured on the test machine: Node lost approximately **half of all new TLS connections**, each failing at a ten-second connect timeout, sequential and concurrent alike, while `cur1` succeeded four times in five. The link is NAT64 and drops connections in bursts. In practice five of six drivers failed to register and the survivor advanced one simulation tick every forty-five seconds.

Decision. The runs were discarded. Figures from them would describe the office connection, not the product, and publishing them would be worse than publishing nothing. The result of 9 August 2026 — five defects found in three and a half minutes, then no failures across 400 API calls — remains the standing measurement, and it was taken on a sound connection.

Incident: eight simulated accounts left behind

The first attempt was terminated mid-run by an operator timeout, stranding its accounts in the test project. This is the documented failure mode for an interrupted run. Recovery was performed and the project is a disposable test instance holding no real data; no production system was involved at any point.

5 • Defect found in our own tooling

POP-001 • `clean` reported a false all-clear • Severity: high • Fixed

What happened. The cleanup loop caught every error and counted it as an account that no longer existed. A network failure therefore produced the message *"0 removed, 8 not found (already gone)"* while the accounts were still present — proved by a direct password sign-in that succeeded moments later.

Why it matters. `clean` is the mechanism that removes invented people from a customer's database. A false all-clear there is the precise failure this product exists to prevent, and it sat on the safety path.

Fix. Transport failures are now separated from genuine absences, listed by name, and the command exits non-zero so neither a human nor a CI job is told the environment is clear when it was never checked.

5.1 A second, related defect

Investigating the first exposed a second: `clean` reached each account through `createUser`, which *signs up* when an identity is absent. Cleaning an already-clean environment therefore created every identity and deleted it again — writing to a customer's authentication table purely to prove that table was empty, and making the per-account result useless as evidence of what had been stranded.

An optional cleanup capability, `signIn`, now lets an adapter answer *"does this identity exist?"* without creating it. It is deliberately **not** a fourteenth contract method: coverage measures how much of a customer's application a run exercises, and a cleanup-only capability does not belong in that denominator. The published thirteen-method contract is unchanged. Four self-tests cover both an adapter that implements the capability and one that does not.

A candour note. During this investigation the author briefly cited the cleanup output as proof the first defect was real; it was not proof, because the command may have created the very accounts it reported removing. The sound evidence was the direct sign-in probe. The claim was withdrawn and the misleading wording removed. It is recorded here because a report that hides its own corrections is the kind of report this company does not write.

6 • Conclusion

- **Buzz Buzz** builds cleanly, its schema applies in full to a fresh environment, and its access controls hold against an anonymous caller holding the shipped key.
- **Populace** passes its full suite, finds a planted defect end to end, and refuses a production target when instructed to attack one.
- **One live check remains outstanding.** It is blocked on network conditions, not on either product, and the procedure to complete it is recorded.
- **One defect was found and fixed** in Populace, in the code that protects a customer's environment — found precisely because the tool was pointed at a real one.